第十章 异常处理

本章内容

- 异常的基本概念
- C++的异常处理机制

异常概述

程序的错误通常包括：

- 语法错误。指程序的书写不符合语言的语法规则。这类错误可由编译程序发现。
- 逻辑错误。指程序设计不当造成程序没有完成预期的功能。这类错误通过测试发现。
- 运行异常。指程序设计对程序运行环境考虑不周全而造成的程序在运行中异常终止，如：内存空间不足、打开不存在的文件进行读操作、程序执行了除以0的指令等等。

- 例如：

```
void f(char *filename)
{  ifstream file(filename);
   int x;
   file >> x; //如果filename指定的文件不
                  //存在，将会出现运行异常!
   ......
}
```

- 程序异常错误往往是由于程序设计者对程序运行环境的一些特殊情况考虑不足所造成的。
- 导致程序运行异常的情况是可以预料的，但它是无法避免的。
- 为了保证程序的鲁棒性，必须在程序中对可能的异常进行预见性处理。

处理异常的策略

- 就地处理（在发现错误的地方处理异常）
  - 调用C++标准库中的函数exit或abort终止程序执行（在cstdlib或stdlib.h中声明）
    - abort立即终止程序的执行，不作任何的善后处理工作；
    - exit在终止程序的运行前，会做关闭被程序打开的文件、调用全局对象和static存储类的局部对象的析构函数（注意：不要在这些对象的析构函数中调用exit）等工作。
  - 根据不同的情况给出不同的处理。

- 异地处理（在非异常发现地处理异常）
  - 通过函数的返回值或指针/引用类型的参数把异常情况通知函数的调用者，由调用者处理。
    - 程序的可读性差！
    - 程序的正常处理与异常处理混杂在一起
  - 通过语言提供的结构化异常处理机制进行处理。

C++异常处理机制

- 把有可能遭遇异常的一系列操作（语句或函数调用）构成一个try语句块。
- 如果try语句块中的某个操作在执行中发现了异常，则通过执行一个throw语句抛掷（产生）一个异常对象。
- 抛掷的异常对象将由能够处理这个异常的地方通过catch语句块来捕获并处理之。

try语句

- try语句块的作用是启动异常处理机制。格式：

```
try
{ <语句序列>
}
```

- 上述的<语句序列>中可以有函数调用。例如，在调用函数f的函数中，可把对f的调用放在一个try语句块中：

```
try
{ f(filename);
}
```

throw语句

- throw语句用于在发现异常情况时抛掷（产生）异常对象。
  throw <表达式>; <表达式>为任意类型的C++表达式（void除外）。

例如:

```
void f(char *filename)
{ ifstream file(filename);
  if (file.fail())
  { throw filename; //产生异常对象（一个字符串指针）
  }
  ......
}
```

- 执行throw语句后，接在其后的语句将不再继续执行，而是转向异常处理（由某个catch语句给出）。

catch语句

- catch语句块紧接在某个try语句的后面，用于捕获throw抛掷的异常对象并处理相应的异常。

  catch (<类型> [<变量>])

  { <语句序列>

  }

  - <类型>用于指出捕获何种异常对象，它与异常对象的类型匹配规则同函数重载；
  - <变量>用于存储异常对象，它可以缺省，缺省时表明catch语句块只关心异常对象的类型，而不考虑具体的异常对象。

例如：

```
char filename[100];
cout <<“请输入文件名：” <<endl;
cin >> filename;
try
{  f(filename);//如果在函数f中抛掷了char *类型的异常，
                //则程序转到try后面的catch(char *str)处理。
}
catch (char *str)
{  cout << str <<“不存在!”<< endl;
   cout <<“请重新输入文件名：” <<endl;
   cin >> filename;
   f(filename);
}
```

一个try语句块的后面可以跟多个catch语句块，用于捕获不同类型的异常对象并进行处理。例如：

```
int g()
{ ......
  try
  { f();
  }
  catch (int) //处理函数f中的throw 1;
  { <语句序列1>}
  catch (double) //处理函数f中的throw 1.0
  { <语句序列2>}
  catch (char *) //处理函数f中的throw
"abcd"
  { <语句序列3>}
  <非catch语句>
}
```

```
void f()
{ ......
  ...throw 1;
  ......
  ...throw 1.0;
  ......
  ...throw "abcd";
  .......
}
```

关于try、throw和catch几点注意

- 在try语句块的<语句序列>执行中如果没有抛掷（throw）异常对象，则其后的catch语句不执行，而是继续执行try语句块之后的非catch语句。
- 在try语句块的<语句序列>执行中如果抛掷（throw）了异常对象，
  - 该try语句块之后有能够捕获该异常对象的catch语句，则执行这个catch语句中的<语句序列>，然后继续执行这个catch语句之后的非catch语句。
  - 该try语句块之后没有能够捕获该异常对象的catch语句，则由函数调用链上的上一层函数中的try语句的相应catch来捕获。

- 如果抛掷异常对象的throw语句不是由程序中的某个try语句块中的<语句序列>调用的，则抛掷的异常不会被程序中的catch捕获。

异常处理的嵌套

- try语句是可以嵌套的。
- 当在内层的try语句的执行中产生了异常，则首先在内层try语句块之后的catch语句序列中查找与之匹配的处理，如果内层不存在能捕获相应异常的catch，则逐步向外层进行查找。
- 如果抛掷的异常对象在程序的调用链上没有给出捕获，则调用系统的terminate函数进行标准的异常处理。默认情况下，terminate函数将会去调用abort函数。

```
void f()
{ try
  { g();
  }
  catch (int)
  {        ......
  }
  catch (char *)
  { ......
  }
}
```

```
void g()
{ try
  { h();
  }
  catch (int)
  { ......
  }
}
```

```
void h()
{    ......
     ... throw 1; //由g捕获并处理
     ......
     ... throw "abcd"; //由f捕获并处理
     ......
}
```

例：一种处理除数为0的异常错误

```
#include<iostream>
using namespace std;
int divide(int x, int y)
{ if (y == 0) throw 0;
  return x/y;
}
void f()
{    int a,b;
     try
     {       cout << "请输入两个数：";
             cin >> a >> b;
             int r=divide(a,b);
             cout << a << "除以" << b << "的商为：" << r << endl;
     }
     catch(int)
     {       cout << "除数不能为0，请重新输入两个数：";
             cin >> a >> b;
             int r=divide(a,b);
             cout << a << "除以" << b << "的商为：" << r << endl;
     }
}
```

```
int main()
{   try
    { f();
    }
    catch (int)
    {    cout << "请重新运行本程序！ "<< endl;
    }
    return 0;
}
```